GUIDE 01

Deploy LangGraph Agents to Railway App

A complete step-by-step guide with common fixes and production tips

Agentic AI Builder Expert Bootcamp

Batch 4.0 | April 2026

Table of Contents

1. Prerequisites and Environment Setup

Before deploying a LangGraph agent to Railway, ensure you have the following tools and accounts configured.

Required Accounts

- Railway account (railway.app) with a project created
- GitHub or GitLab account for source code hosting
- OpenAI / Anthropic / Azure API key for LLM calls

Local Tools

- Python 3.11+ installed locally
- Docker Desktop installed and running
- Railway CLI installed

```
npm install -g @railway/cli
railway login
```

Python Dependencies

```
pip install langgraph langchain langchain-openai fastapi uvicorn python-dotenv
```

Tip: *Always pin your dependency versions in requirements.txt for reproducible deployments.*

2. Project Structure for LangGraph Agents

Organize your project with a clear separation between agent logic, API layer, and deployment configs.

```
langgraph-railway/
|-- agent/
|   |-- __init__.py
|   |-- graph.py          # LangGraph agent definition
|   |-- nodes.py          # Individual node functions
|   |-- tools.py          # Tool definitions
|   |-- state.py          # State schema
|-- api/
|   |-- __init__.py
|   |-- main.py           # FastAPI app
|   |-- routes.py         # API routes
|-- Dockerfile
|-- requirements.txt
|-- railway.toml
|-- .env
|-- .gitignore
```

Tip: *Keep the agent/ module independent of the API layer so you can test the graph separately using LangGraph Studio or a local script.*

3. Building the LangGraph Agent

3.1 Define the State Schema

Create a TypedDict that represents the state flowing through your graph. This is the backbone of your agent.

```
# agent/state.py
from typing import TypedDict, Annotated, Sequence
from langchain_core.messages import BaseMessage
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], add_messages]
    next_step: str
    tool_output: str
```

3.2 Define Node Functions

```
# agent/nodes.py
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, AIMessage
from .state import AgentState

llm = ChatOpenAI(model="gpt-4o", temperature=0)

def call_model(state: AgentState) -> dict:
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

def should_continue(state: AgentState) -> str:
    last = state["messages"][-1]
    if hasattr(last, "tool_calls") and last.tool_calls:
        return "tools"
    return "end"

def call_tools(state: AgentState) -> dict:
    # Execute tool calls from the last message
    last = state["messages"][-1]
    # ... tool execution logic
    return {"messages": [...]}
```

3.3 Build the Graph

```
# agent/graph.py
from langgraph.graph import StateGraph, END
from .state import AgentState
from .nodes import call_model, should_continue, call_tools

workflow = StateGraph(AgentState)
workflow.add_node("agent", call_model)
workflow.add_node("tools", call_tools)

workflow.set_entry_point("agent")
workflow.add_conditional_edges(
```

```
    "agent", should_continue,
    {"tools": "tools", "end": END}
)
workflow.add_edge("tools", "agent")

graph = workflow.compile()
```

4. Creating the FastAPI Server

```
# api/main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from langchain_core.messages import HumanMessage
from agent.graph import graph
import os, uvicorn
from dotenv import load_dotenv

load_dotenv()
app = FastAPI(title="LangGraph Agent API")

class QueryRequest(BaseModel):
    message: str
    thread_id: str = "default"

class QueryResponse(BaseModel):
    response: str
    messages_count: int

@app.get("/health")
def health():
    return {"status": "healthy", "service": "langgraph-agent"}

@app.post("/chat", response_model=QueryResponse)
async def chat(req: QueryRequest):
    try:
        inputs = {"messages": [HumanMessage(content=req.message)]}
        result = graph.invoke(inputs)
        final = result["messages"][-1].content
        return QueryResponse(
            response=final,
            messages_count=len(result["messages"])
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 8000))
    uvicorn.run("api.main:app", host="0.0.0.0", port=port)
```

Warning: *Railway dynamically assigns the PORT variable. Always read it from os.environ, never hardcode it.*

5. Dockerizing the Application

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app

# Install dependencies first for caching
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Railway sets PORT dynamically
ENV PORT=8000
EXPOSE $PORT

CMD ["python", "-m", "api.main"]
```

requirements.txt

```
langgraph==0.2.60
langchain==0.3.14
langchain-openai==0.3.0
fastapi==0.115.0
uvicorn[standard]==0.32.0
python-dotenv==1.0.1
```

Tip: *Use python:3.11-slim instead of python:3.11 to reduce image size by ~800MB. This significantly speeds up Railway builds.*

6. Deploying to Railway

Step 1: Initialize Railway Project

```
railway init
```

Step 2: Link to GitHub Repository

Go to Railway Dashboard and click 'New Project' then 'Deploy from GitHub repo'. Select your repository and branch.

Step 3: Configure via railway.toml

```
# railway.toml
[build]
builder = "dockerfile"
dockerfilePath = "Dockerfile"

[deploy]
startCommand = "python -m api.main"
healthcheckPath = "/health"
```

```
healthcheckTimeout = 300
restartPolicyType = "on_failure"
restartPolicyMaxRetries = 5
```

Step 4: Deploy via CLI

```
railway up
```

Step 5: Verify Deployment

```
railway logs
# Or check the health endpoint
curl https://your-app.up.railway.app/health
```

7. Environment Variables and Secrets

Never commit API keys to your repository. Use Railway's built-in secrets manager.

Setting Variables via CLI

```
railway variables set OPENAI_API_KEY=sk-xxx
railway variables set LANGCHAIN_TRACING_V2=true
railway variables set LANGCHAIN_API_KEY=ls-xxx
railway variables set LANGCHAIN_PROJECT=prod-agent
```

Setting Variables via Dashboard

Navigate to your service in Railway Dashboard, click the 'Variables' tab, and add each key-value pair. Railway encrypts all variables at rest.

> ***Warning:*** *If you use LangSmith for tracing, set LANGCHAIN_TRACING_V2=true, LANGCHAIN_API_KEY, and LANGCHAIN_PROJECT in Railway variables.*

8. Custom Domain and Networking

Railway provides a free *.up.railway.app domain. For production, configure a custom domain.

Step 1: Go to Settings > Networking in Railway Dashboard

Step 2: Click 'Generate Domain' for a railway.app subdomain

Step 3: For custom domains, add a CNAME record pointing to your Railway domain

```
# DNS Configuration Example
Type:  CNAME
Name:  api
Value: your-app.up.railway.app
```

9. Common Errors and Fixes

Below are the most frequently encountered errors when deploying LangGraph agents to Railway, along with their solutions.

Error	Cause	Fix
ModuleNotFoundError: langgraph	Missing from requirements.txt	Add langgraph==0.2.x to requirements.txt and rebuild
Port already in use	Hardcoded port conflicts with Railway	Always use os.environ.get('PORT', 8000)
Health check failed	No /health endpoint or slow startup	Add a /health route and increase healthcheckTimeout in railway.toml to 300+
Container exits immediately	No blocking process running	Ensure uvicorn.run() is called with host='0.0.0.0'
OPENAI_API_KEY not set	Env var missing in Railway	Set via railway variables set OPENAI_API_KEY=sk-xxx
Build timeout	Large dependencies or slow build	Use Docker layer caching - COPY requirements.txt first, then pip install
Memory exceeded (OOMKilled)	LangGraph state too large	Upgrade Railway plan or optimize state; limit message history length
SSL certificate error	Outbound HTTPS issues	Ensure python certifi is installed; add 'certifi' to requirements.txt
ImportError: pydantic	Version conflict between LangChain and Pydantic	Pin pydantic>=2.0,<3.0 in requirements.txt
Recursion limit reached	Agent stuck in loop	Set recursion_limit in graph.invoke(): graph.invoke(inputs, {'recursion_limit': 25})

Fix: Dockerfile Build Failures

```
# If pip install fails, try:
RUN pip install --no-cache-dir --upgrade pip setuptools wheel
RUN pip install --no-cache-dir -r requirements.txt

# If native dependencies are needed:
RUN apt-get update && apt-get install -y \
    gcc g++ && \
    rm -rf /var/lib/apt/lists/*
```

Fix: LangGraph Checkpoint Persistence

Railway containers are ephemeral. If you need persistent checkpoints (for multi-turn conversations), use an external store.

```
# Use PostgreSQL checkpoint saver
from langgraph.checkpoint.postgres import PostgresSaver
import os

DB_URL = os.environ["DATABASE_URL"]
checkpointer = PostgresSaver.from_conn_string(DB_URL)
graph = workflow.compile(checkpointer=checkpointer)
```

Tip: *Railway offers a PostgreSQL plugin. Add it to your project and Railway auto-injects DATABASE_URL.*

10. Health Checks and Monitoring

Configure comprehensive health checks for production reliability.

```
@app.get("/health")
async def health():
    checks = {"api": "ok"}
    try:
        # Test LLM connectivity
        from langchain_openai import ChatOpenAI
        llm = ChatOpenAI(model="gpt-4o-mini")
        llm.invoke("ping")
        checks["llm"] = "ok"
    except Exception as e:
        checks["llm"] = f"error: {str(e)}"
    status = "healthy" if all(
        v == "ok" for v in checks.values()
    ) else "degraded"
    return {"status": status, "checks": checks}
```

11. Scaling and Production Tips

- **Use async endpoints:** FastAPI supports async natively. Use *async def* for all LLM-calling routes to handle concurrent requests.
- **Rate limiting:** Add slowapi or a custom middleware to prevent abuse. LLM calls are expensive.
- **Logging:** Use structured logging with the *structlog* library. Railway captures stdout/stderr automatically.
- **Auto-scaling:** Railway supports horizontal scaling. Enable replicas in your service settings for high-traffic agents.
- **Caching:** Cache LLM responses for repeated queries using Redis (available as a Railway plugin).
- **Timeouts:** Set request timeouts in uvicorn (--timeout-keep-alive 120) to handle long-running agent chains.
- **CI/CD:** Railway auto-deploys on git push. Use branch-based environments (staging from 'develop', production from 'main').

Summary: LangGraph + FastAPI + Docker + Railway gives you a robust, scalable deployment pipeline for agentic AI. Always test locally with Docker first, use Railway's variable management for secrets, and implement health checks for reliability.